

Arduino IR-Controlled & Obstacle-Avoidance Robot

Engineering Design, Implementation, Control Logic and Reproduction Guide

Author: Hailemariam Muche

Discipline: Electrical and Computer Engineering

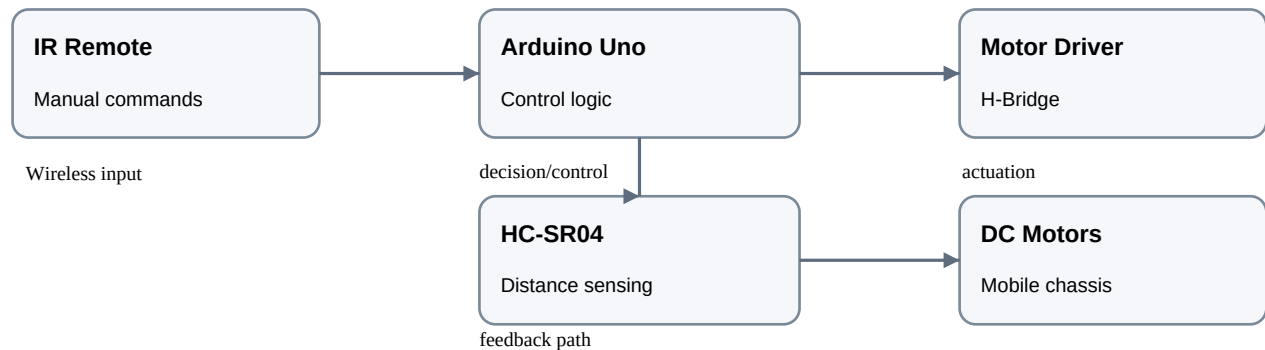
Repository:

<https://github.com/Hailemariam-Muche/arduino-ir-controlled-and-obstacle-avoidance-robot>

Document purpose. This document provides an engineering-level record of the robot design, hardware organization, embedded control strategy, software structure, testing considerations, and repository contents. It is intended to make the project understandable and reproducible without relying solely on the source code.

1. Project Overview

This project develops a compact mobile robot around the Arduino platform and demonstrates two complementary control approaches: manual wireless navigation using an infrared remote and autonomous movement with ultrasonic obstacle detection. The repository currently describes separate firmware configurations for an IR-controlled H-bridge drive and an autonomous continuous-rotation-servo configuration. This separation is deliberate: it allows the same platform to be used to demonstrate both direct command execution and sensor-driven decision making.



The project is particularly valuable as an engineering teaching and prototyping platform because it brings together microcontroller programming, digital I/O, PWM-based actuation, infrared communication, ultrasonic ranging, motor control, and embedded decision logic in one physical system.

2. Engineering Objectives

- Design and implement a functional Arduino-based mobile robotic platform.
- Provide reliable directional control from a standard infrared remote transmitter.
- Decode predefined IR commands and translate them into motor-control actions.
- Measure the distance to obstacles using an HC-SR04 ultrasonic sensor.
- Implement a basic autonomous escape strategy when the measured distance enters a collision-risk region.
- Provide simulation and supporting library resources so the design can be studied and reproduced.
- Document the relationship between the electrical interface, embedded firmware, and mechanical motion.

3. System Architecture

The repository contains two hardware/firmware configurations. The first uses continuous-rotation servo motors with ultrasonic sensing for autonomous operation. The second uses two geared DC motors driven through an H-bridge and an IR receiver for manual control. The configurations share the Arduino control concept but use different actuator interfaces.

3.1 Configuration A — Autonomous ultrasonic/servo platform

Interface	Arduino pin	Role
HC-SR04 Trigger	D7	Ultrasonic trigger pulse
HC-SR04 Echo	D8	Echo-time measurement
Right continuous servo	D9	Right-side drive
Left continuous servo	D10	Left-side drive

3.2 Configuration B — IR-controlled DC motor platform

Interface	Arduino pin	Role
Right motor IN4	D4	Right motor phase/control
Right motor IN3	D5	Right motor phase/control
Left motor IN2	D7	Left motor phase/control
Left motor IN1	D8	Left motor phase/control
IR receiver OUT	D10	IR data input
Status LED	D13	Activity indication

4. Hardware Design

The repository identifies the Arduino Uno R3 (ATmega328P) as the principal controller. For the autonomous configuration, an HC-SR04 ultrasonic sensor and two continuous-rotation servos form the sensing and drive system. For the manual configuration, an infrared demodulator receives the remote signal and an L298N dual H-bridge provides the electrical interface to two geared DC motors.

Component	Engineering function
Arduino Uno R3 / ATmega328P	Real-time control, sensor acquisition and command processing
HC-SR04	Non-contact distance measurement
TSOP382 / VS1838B	Demodulation of IR remote signals
L298N dual H-bridge	Bidirectional control of two DC motors
2 × continuous-rotation servos	Alternative autonomous drive arrangement
2 × geared DC motors	Manual-control drive arrangement
IR handheld remote	Wireless operator input
Battery supply	Portable energy source for the mobile platform

5. Infrared Control Protocol

The manual-control firmware recognizes specific hexadecimal command values and maps each command to a motion primitive. This makes the control layer deterministic and easy to verify with a serial monitor or equivalent diagnostic output.

IR code	Command	Motion
0xFF02FD	Forward	Both drive channels commanded forward
0xFF9867	Backward	Both drive channels commanded in reverse
0xFFE01F	Left	Differential/turning action
0xFF906F	Right	Differential/turning action
0xFFA857	Stop	Motor-control outputs stopped/braked

The repository includes IR library resources, including an IRremote package archive. When reproducing the project, the library version and API must match the firmware source being compiled; IRremote APIs have changed between major releases.

6. Ultrasonic Obstacle-Avoidance Algorithm

The autonomous firmware generates a short trigger pulse, measures the returning echo duration, and converts the elapsed time into distance. The repository expresses the calculation in inches as: $\text{Distance} = \text{Echo duration} / 74 / 2$. The resulting value is compared with a safety threshold.

Condition	Controller response
Distance $\geq$ 5 in	Continue normal motion/tracking
Distance $<$ 5 in	Stop, reverse, then resume tracking

The documented escape sequence uses a stop period of approximately 2 seconds followed by reverse motion for approximately 2 seconds. This is a deliberately simple reactive strategy: it is easy to understand and implement, but it does not construct a map or perform predictive path planning.

7. Firmware Structure

The repository separates the principal demonstrations into dedicated Arduino sketches. The manual-control sketch is responsible for IR reception, command recognition and motor-control functions. The obstacle-avoidance sketch is responsible for ultrasonic timing, distance calculation and autonomous movement.

- Initialize I/O and actuator interfaces before entering the main control loop.
- Continuously poll the appropriate input source.
- Convert raw sensor/receiver data into a meaningful engineering quantity or command.
- Apply the corresponding motion/control function.
- Maintain a simple, deterministic control loop suitable for a small microcontroller.

8. Simulation and Supporting Resources

The GitHub repository includes Arduino and Proteus-related library resources, IR receiver/Proteus resources, the IRremote archive, and the main source-code directory. These resources support both educational simulation and hardware-oriented development. A reproduction should install only the libraries required by the selected sketch and should verify pin assignments before running the simulation or applying power to the physical circuit.

9. Development and Test Procedure

Stage	Verification activity
1. Firmware compilation	Confirm board type, selected sketch and required libraries.
2. IR receiver test	Observe decoded values and compare them with the expected command table.
3. Motor test	Verify left/right polarity and forward/reverse direction at low power.
4. Ultrasonic test	Measure known distances and compare calculated values.
5. Autonomous test	Introduce an obstacle and verify stop/reverse behavior.
6. Integrated test	Run the complete system on a controlled surface and inspect stability, power and response.

10. Engineering Considerations

- Motor current can exceed the safe current available from a microcontroller supply; motors should therefore be powered through the appropriate driver and a suitable battery source.
- All logic and motor-control electronics should share a suitable reference ground while respecting the electrical limits of the controller.
- Ultrasonic readings can be affected by surface angle, acoustic reflections and mechanical vibration; threshold decisions should therefore include practical safety margin.
- IR performance depends on receiver alignment, ambient infrared interference and the characteristics of the remote transmitter.
- The 5-inch obstacle threshold is an implementation parameter, not a universal safety distance. It should be recalibrated for chassis speed, stopping distance and environment.

11. Repository Organization

The current GitHub repository provides a source-code directory together with Proteus/IR support archives and the project README. The repository is the authoritative location for the actual firmware and supporting files.

Repository item	Purpose
Source_Code/	Arduino firmware and source implementations
Arduino Library for Proteus.rar	Arduino simulation support
IRX-Proteus-1.0.zip / IRX_Library.zip	IR receiver/Proteus support resources
IRremote-4.7.1.zip	IR decoding library archive included in repository
Ultrasonic-Sensor-HC-SR04-Proteus-Library.zip	Ultrasonic simulation library
README.md	Project summary, wiring and usage information

12. Results and Engineering Value

The completed project demonstrates a practical transition from low-level digital signals to physical robotic behavior. The IR path illustrates deterministic human-in-the-loop control, while the ultrasonic path illustrates closed-loop sensing and reactive autonomy. The combination provides a compact platform for teaching embedded systems, robotics, sensor interfacing and actuator control.

13. Limitations and Future Development

- Replace the single-threshold escape behavior with multi-direction scanning using a servo-mounted ultrasonic sensor.
- Add speed control using PWM and closed-loop wheel feedback.
- Add a formal finite-state machine for manual, autonomous, stop and fault states.
- Add battery-voltage monitoring and low-power protection.
- Add encoder feedback for repeatable turning and distance estimation.
- Expand obstacle avoidance from reactive reversal to left/right path evaluation.
- Add structured serial diagnostics for easier field testing.

14. Reproduction Checklist

- Obtain the repository and identify the intended firmware configuration.
- Install the required Arduino board support and matching libraries.
- Verify every pin against the selected sketch before wiring.
- Test the receiver/sensor independently before connecting the drive system.
- Use a suitable motor power source and common electrical reference.
- Perform initial tests with the robot lifted or on a safe test stand.
- Only after directional control and stopping behavior are verified, perform floor testing.

15. Repository Reference

Primary GitHub repository:

<https://github.com/Hailemariam-Muche/arduino-ir-controlled-and-obstacle-avoidance-robot>

This document is intended to complement the repository rather than replace the source code. Firmware, simulation libraries and project files should always be taken from the repository version being used for reproduction.

Prepared as an engineering project record

Hallemariam Muche — Electrical and Computer Engineering